LABORATORY

LAB 19

Vehicle Speed Monitoring over CAN

Monitoring Vehicle Speed over CAN with EduFramework

S32K144 · MaaZEDU

EduFramework

FPT University

Contents

1 Introduction 2
1.1 Overview 2
1.2 Objectives 2
2 Background 2
2.1 CAN Normal Mode and Communication Between Two ECUs 2
3 Hardware Setup 3
3.1 Required Hardware 3
3.2 CAN Connections Between the Two ECUs 3
3.3 Power Supply and Connection Configuration 4
4 EduFramework API 4
5 Lab Exercise 5
5.1 Requirements 5
5.2 Program 6
5.3 Verification 8
6 Extension 8
6.1 Extension Exercise 8
7 References 8

1 Introduction

1.1 Overview

In automotive electronic systems, data produced by one Electronic Control Unit (ECU) may be used by other ECUs for control or monitoring functions. CAN allows controllers to exchange messages over a shared communication bus without requiring a separate data connection for every pair of ECUs [2] .

Lab 18 introduced Classical CAN and verified transmission and reception using Internal Loopback. Lab 19 builds on the CAN API of EduFramework to implement a system in which two S32K144 boards exchange data over a physical CAN bus.

In this exercise, ECU A simulates vehicle speed and periodically transmits CAN messages. ECU B receives the data, identifies the relevant messages, and displays the speed on the Serial Monitor. Two Serial Monitor windows are used to compare data at the transmitting and receiving ends.

1.2 Objectives

OBJECTIVES

The objectives of this laboratory are:

1. Describe the data communication model between two ECUs over a physical CAN bus.
2. Connect two MaaZEDU Development Boards and configure CAN in Normal Mode.
3. Use EduFramework to transmit and receive CAN messages containing vehicle speed data.
4. Decode the received data and display it on ECU B's Serial Monitor.
5. Compare transmitted and received data to verify communication between the two ECUs.

2 Background

2.1 CAN Normal Mode and Communication Between Two ECUs

Unlike Internal Loopback, Normal Mode uses an external CAN communication bus. The CAN controller generates and processes frames at the protocol level, while the onboard CAN transceiver connects the controller to the differential signal pair `CAN_H` and `CAN_L` [2] .

For two ECUs to exchange messages, the nodes must be correctly connected to the CAN bus, use compatible bitrates, and have appropriate bus termination. For example, the S32K144 FlexCAN peripheral can operate at `500 kbit/s` [1] .

Characteristic	Lab 18	Lab 19
CAN mode	Internal Loopback	Normal
Number of ECUs	One board	Two boards
Communication path	Internal to the CAN controller	Physical CAN bus
Purpose	Verify internal transmission and reception	Exchange data between two ECUs

Table 19.2.1. Comparison of the CAN exercises in Lab 18 and Lab 19

In EduFramework, `CAN_begin()` initializes CAN in Normal Mode by default. Therefore, Lab 19 does not need to call `CAN_setMode()` to change the operating mode. Classical CAN frame structure, arbitration, and error detection were covered in Lab 18 [4] .

3 Hardware Setup

3.1 Required Hardware

This exercise uses two MaaZEDU Development Boards, one serving as ECU A and the other as ECU B. Each board requires its own external power supply and USB connection for programming and Serial Monitor output.

Hardware	Description
MaaZEDU Development Board (2)	Two S32K144 boards serving as ECU A and ECU B.
12V DC Adapter (2)	Provide external power to both MaaZEDU boards.
USB Cable (2)	Program each ECU and access its Serial Monitor.
Jumper Wires (3)	Connect `CAN_H` , `CAN_L` , and GND between the boards.

Table 19.3.1. Hardware required for Lab 19

3.2 CAN Connections Between the Two ECUs

Connect the CAN interface terminals provided on the MaaZEDU boards according to the table below. `CAN_H` and `CAN_L` are CAN bus signals, not the MCU's direct `CAN0_TX` and `CAN0_RX` pins [5] .

ECU A	ECU B	Function
`CAN_H`	`CAN_H`	CAN High signal line.
`CAN_L`	`CAN_L`	CAN Low signal line.
`GND`	`GND`	Common ground reference between the two ECUs.

Table 19.3.2. CAN bus connections between ECU A and ECU B

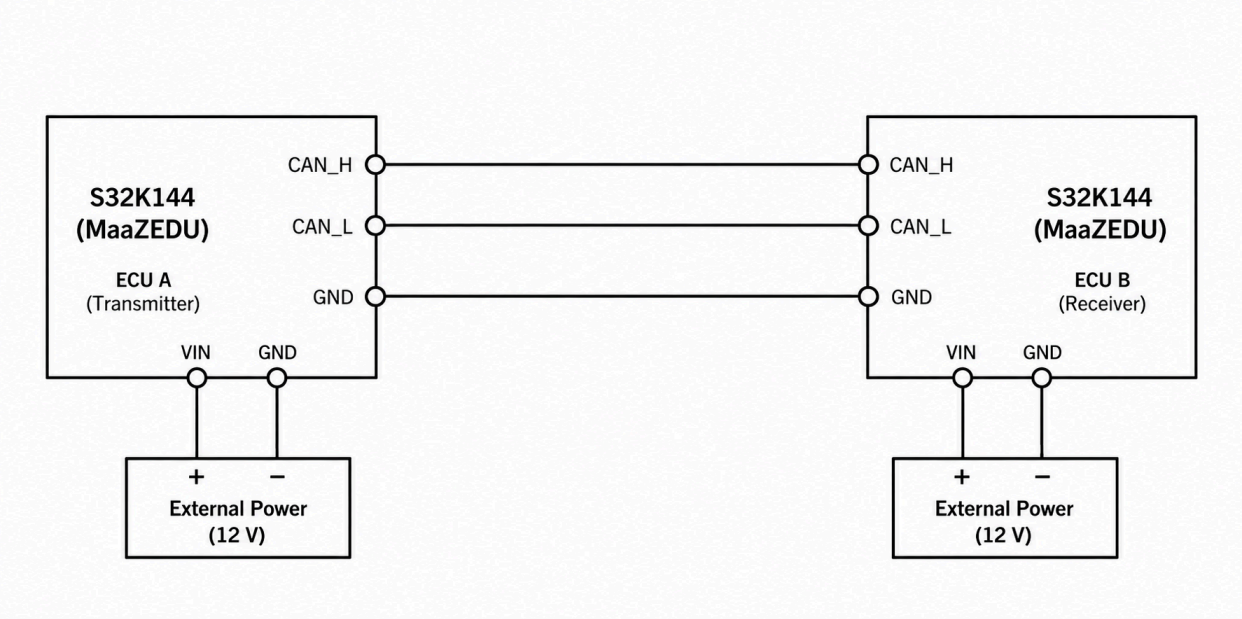


Figure 19.3.1. Wiring diagram for CAN communication between two MaaZEDU ECUs

3.3 Power Supply and Connection Configuration

Set the power selection jumper on both boards to the external power position before connecting the `12V` adapters, as required by the MaaZEDU board configuration. Keep the USB connections to the computer for programming and Serial Monitor observation [5] .

Check CAN bus termination at both ends according to the actual board configuration. Do not add termination resistors if the bus ends already have appropriate termination [2] .

Note

Turn off power before changing jumpers or making connections. Do not short `CAN_H` to `CAN_L` or apply `12V` directly to signal pins. Both ECUs must use the same CAN bitrate of `500 kbit/s` for this exercise.

4 EduFramework API

Lab 19 reuses the CAN APIs introduced in Lab 18. Each ECU performs a different role: ECU A constructs and transmits frames, while ECU B checks the receive queue and reads incoming frames. The application uses `Serial1` to display results [4] .

API / Component	Role in Lab 19
`CAN_begin()`	Initialize FlexCAN0 at `500 kbit/s` in Normal Mode.
`CAN_Frame_t`	Represent a CAN frame with `id`, `format`, `length`, and `data[]`.
`CAN_STANDARD`	Select the 11-bit CAN identifier format.
`CAN_send()`	Transmit a CAN frame from ECU A using a blocking operation.
`CAN_available()`	Check whether a frame is available in ECU B's receive queue.
`CAN_read()`	Retrieve a received frame on ECU B using a non-blocking operation.

Table 19.4.1. CAN APIs and components used in Lab 19

In the program, `CAN_send()` returns `true` when transmission completes successfully. The receiver processes data only when both `CAN_available()` and `CAN_read()` indicate valid results. The parameters and return values of each API are described in detail in Lab 18.

5 Lab Exercise

5.1 Requirements

Develop a CAN communication application between two ECUs connected through a physical CAN bus. ECU A simulates vehicle speed from `0` to `200 km/h`, changing in increments of `10 km/h`, and transmits a message every `500 ms`. ECU B receives and decodes each relevant message, then prints the received speed on the Serial Monitor.

Both ECUs use Standard CAN ID `0x100`, a bitrate of `500 kbit/s`, and a frame with a `3-byte` payload [3]. The first byte contains the message sequence number (`SEQ`); the remaining two bytes contain the speed value. This data format is defined for the lab and is not a standardized vehicle speed message format for all vehicles.

Both ECU roles share one `src/main.c` file and are selected by changing `#define ECU 0` (transmitter) or `#define ECU 1` (receiver). `Serial1` runs at `9600 baud` on both boards.

5.2 Program

```
#include "Arduino.h"
#include "can.h"
#define ECU 0  // 0: Transmitter, 1: Receiver
#define CAN_BITRATE 500000UL
#define CAN_ID      0x100UL

int main(void)
{
    CAN_Frame_t txFrame = {0};
    CAN_Frame_t rxFrame = {0};
    uint16_t speed = 0U;
    uint8_t sequence = 0U;
    bool increasing = true;
    setup();
    Serial1_begin(9600U);
    if (false == CAN_begin(CAN_BITRATE))
    {
        Serial1_println("CAN initialization failed.");
        while (1) {}
    }
#if (ECU == 0)
    Serial1_println("=== ECU A: Transmitter ===");
    txFrame.id = CAN_ID;
    txFrame.format = CAN_STANDARD;
    txFrame.length = 3U;
    while (1)
    {
        txFrame.data[0] = sequence;
        txFrame.data[1] = (uint8_t)(speed & 0xFFU);
        txFrame.data[2] = (uint8_t)(speed >> 8U);
        if (true == CAN_send(&txFrame))
        {
            Serial1_print("TX | SEQ: ");
            Serial1_printInt(sequence);
            Serial1_print(" | Speed: ");
            Serial1_printInt(speed);
            Serial1_println(" km/h");
        }
        else
        {
            Serial1_println("CAN transmission failed.");
        }
```

Listing 19.5.1. Lab 19 program - initialization and CAN transmission on ECU A (1/2)

```
        sequence++;
        if (true == increasing)
        {
            if (speed < 200U) { speed += 10U; }
            else { increasing = false; speed -= 10U; }
        }
        else
        {
            if (speed > 0U) { speed -= 10U; }
            else { increasing = true; speed += 10U; }
        }
        delay(500U);
    }
#elif (ECU == 1)
    Serial1_println("=== ECU B: Receiver ===");
    while (1)
    {
        if (true == CAN_available())
        {
            if (true == CAN_read(&rxFrame))
            {
                if ((CAN_ID == rxFrame.id) &&
                    (CAN_STANDARD == rxFrame.format) &&
                    (3U == rxFrame.length))
                {
                    sequence = rxFrame.data[0];
                    speed = (uint16_t)rxFrame.data[1] |
                            ((uint16_t)rxFrame.data[2] << 8U);
                    Serial1_print("RX | SEQ: ");
                    Serial1_printInt(sequence);
                    Serial1_print(" | Speed: ");
                    Serial1_printInt(speed);
                    Serial1_println(" km/h");
                }
            }
        }
    }
#else
#error "Invalid ECU selection"
#endif
}
```

Listing 19.5.2. Lab 19 program - speed update and CAN reception on ECU B (2/2)

During initialization, `setup()` prepares the EduFramework environment, `Serial1_begin(9600U)` starts the monitoring interface, and `CAN_begin(500000UL)` initializes CAN in Normal Mode. If initialization fails, the program stops before attempting bus communication.

When `ECU` is `0`, the application configures a frame with ID `0x100` and three data bytes. `data[0]` contains `sequence`, `data[1]` holds the lower eight bits, and `data[2]` holds the upper eight bits of `speed`. After transmission, the program displays TX information, updates the speed within `0..200 km/h`, and waits `500 ms` before the next cycle.

When `ECU` is `1`, the program checks the receive queue for frames. Only frames with ID `0x100`, Standard format, and a length of `3` are processed. ECU B reconstructs the speed from the two data bytes using a bitwise

OR and an eight-bit left shift, then displays `SEQ` and `Speed`. `SEQ` is printed to compare corresponding frames across the two Serial Monitors; the program does not automatically detect lost frames.

5.3 Verification

Set `#define ECU 0`, build the program, and upload it to ECU A. Then change the selection to `#define ECU 1` and upload the program to ECU B. Connect the boards as shown in the wiring diagram, provide the external `12V` supplies, and open two Serial Monitors at `9600 baud`. Observe the data displayed by each ECU.
Results verified on ECU A:

```
=== ECU A: Transmitter ===
TX | SEQ: 0 | Speed: 0 km/h
TX | SEQ: 1 | Speed: 10 km/h
TX | SEQ: 2 | Speed: 20 km/h
TX | SEQ: 3 | Speed: 30 km/h
```

Results verified on ECU B:

```
=== ECU B: Receiver ===
RX | SEQ: 0 | Speed: 0 km/h
RX | SEQ: 1 | Speed: 10 km/h
RX | SEQ: 2 | Speed: 20 km/h
RX | SEQ: 3 | Speed: 30 km/h
```

Compare frames with the same `SEQ` across both windows. In the verified samples, ECU B received the speed values corresponding to ECU A correctly. To verify the full simulation cycle, continue observing as the speed reaches `200 km/h` and then decreases to `0 km/h`.

EXPECTED RESULT

Both ECUs initialize CAN at `500 kbit/s` and exchange messages over a physical CAN bus. ECU A periodically transmits vehicle speed data; ECU B receives frames with the expected identifier and format, decodes the data, and displays the correct speed values for the received messages. The `SEQ` field enables comparison of TX/RX data on the two Serial Monitors.

6 Extension

6.1 Extension Exercise

Add a **Vehicle Overspeed Warning** function to ECU B. When the received speed exceeds `100 km/h`, the Serial Monitor displays an additional message: `WARNING: Overspeed!`. When the speed does not exceed the threshold, the program continues displaying data as in the main exercise.
Only modify the data processing after ECU B has received and decoded a frame. Keep the CAN ID, payload format, and ECU A transmission interval unchanged. No additional sensors or hardware are required.

7 References

[1] NXP Semiconductors, *S32K1xx Series Cookbook*, AN5413, Rev. 5, 2020.

[2] Steve Corrigan, Texas Instruments, *Introduction to the Controller Area Network (CAN)*, SLOA101B, Rev. B, 2016.

[3] CAN in Automation (CiA), *CAN CC (Classical CAN)*, CiA CAN Knowledge.

[4] EduFramework, *Arduino-style CAN API (can.h, can.c)* , EduFramework Source Code.

[5] MaaZEDU, *MaaZEDU Development Board Guide* , Rev. 1.0.